

Performance Testing

Date	30 June 2026
Team ID	XXXXXX
Project Name	Rising Waters
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Flask Test Environment, Jupyter Notebook (Model Testing)
Type of Testing	Performance Testing
Target Module / API	Flood Prediction API (Flask Web Application)
Test Environment	Windows 11, Python 3.x, Flask, XGBoost Model, IBM Cloud (Deployment Ready)
Test Date	01 JULY 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits weather data for flood prediction	1	30	Prediction generated accurately within acceptable response time
2	Multiple users access the flood prediction system simultaneously	25	60	System handles concurrent requests without errors or major delay
3	Continuous prediction requests during peak usage	50	120	Stable performance with consistent response time and no crashes
4	High-load stress test with maximum concurrent users	100	180	Application remains available, with acceptable performance under heavy load

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.35 seconds	Pass	Fast response under normal load
2	Response Time (Max)	< 5 seconds	3.82 seconds	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 20 Req/sec	28 Req/sec	Pass	Good request handling capacity
4	Error Rate	< 1%	0.2%	Pass	Very low error rate
5	CPU Utilization	< 80%	64%	Pass	CPU usage remained stable
6	Memory Utilization	< 80%	58%	Pass	Memory usage within acceptable range

Step 4: Observations & Analysis

Key Findings:

- The flood prediction system performed efficiently under normal and moderate workloads.
- Average response time remained below 2 seconds, with a very low error rate.
- The XGBoost model delivered 96.55% prediction accuracy, making the system reliable for early flood warning.

Bottlenecks Identified:

- Slight increase in response time was observed during high concurrent user load.
- No major performance bottlenecks or system failures were detected during testing.

Optimization Steps Taken:

- Optimized the XGBoost model for faster prediction.
- Reduced unnecessary data processing and optimized Flask API requests.
- Improved resource management to lower CPU and memory usage.
- Configured the application for scalable deployment on IBM Cloud.

Step 5: Screenshots / Evidence

Step 5: Screenshots / Evidence

[Home](#)[Analytics](#)[Contact](#)

 **Flood Prediction**

Temperature

Humidity

Cloud Cover

Annual Rainfall

Jan-Feb Rainfall

Mar-May Rainfall

Jun-Sep Rainfall

Oct-Dec Rainfall

Average June Rainfall

Sub

Predict Flood Risk

HOMEPAGE/GET STARTED PAGE

[Home](#)[Analytics](#)[Contact](#)

 **Flood Prediction**

Temperature

Humidity

Cloud Cover

Annual Rainfall

Jan-Feb Rainfall

Mar-May Rainfall

Jun-Sep Rainfall

Oct-Dec Rainfall

Average June Rainfall

Sub

Predict Flood Risk

 **Flood Risk Detected**

PREDICTION RESULT